



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Ingeniería
En Desarrollo y Gestión de Software

Seguridad en el desarrollo de aplicaciones

Grupo IDGS17

Nombre del alumno:
Jovanny Guadalupe López Cebreros

Matricula:
2023371119

Profesor:
Gerardo Ramírez Villareal

Práctica Configuracion_Logging_Serilog

Reporte Técnico: Configuración de Logging con Serilog (SEGG-U2-P3G-1)

1. Objetivo de la Práctica

El objetivo principal de esta práctica fue configurar Serilog como la plataforma centralizada de logging para la aplicación VulnerableApp, así como validar la correcta generación y distribución de los registros en diferentes destinos: consola, archivo de texto físico y el servidor de monitoreo Seq.

2. Resolución de Retos (Challenges)

Reto 1: Investigación de la función de cada Sink implementado

Durante la configuración en Program.cs, se implementaron tres Sinks (destinos) diferentes para los logs, cada uno con un propósito específico en el ciclo de vida del software:

- **Console Sink:** Su función principal es imprimir los registros directamente en la terminal de ejecución. Es ideal para la etapa de desarrollo local, ya que permite al desarrollador ver la actividad y los errores de la aplicación en tiempo real sin necesidad de abrir herramientas externas.
- **File Sink:** Su función es persistir los registros en un archivo físico de texto (.txt) dentro del servidor. En esta práctica, se configuró con una política de rotación diaria (RollingInterval.Day), lo que garantiza que los logs se almacenen de forma segura para auditorías futuras, asegurando que el tamaño del archivo no crezca descontroladamente.
- **Seq Sink:** Su función es enviar los logs en formato de datos estructurados (JSON) hacia un servidor centralizado (Seq). A diferencia de los archivos de texto, Seq permite indexar las propiedades, filtrar errores por niveles de criticidad y visualizar el comportamiento de la aplicación a través de una interfaz gráfica intuitiva.

Reto 2: Configuración del nivel mínimo de logging

Para evitar que el nivel de registro estuviera acoplado de forma rígida en el código fuente, se delegó esta configuración al archivo `appsettings.json`. Mediante la instrucción `.ReadFrom.Configuration()`, Serilog lee el nodo `"MinimumLevel"`. Esto permitió realizar pruebas dinámicas; al modificar el nivel de `"Default": "Information"` a `"Default": "Debug"`, se comprobó que la aplicación comenzó a emitir trazas mucho más detalladas del funcionamiento interno del framework (como el enrutamiento de MVC) sin necesidad de recompilar el proyecto.

Reto 3: Enriquecedor adicional y su justificación

Además del enriquecedor de contexto estándar, se agregó el enriquecedor `WithMachineName()`.

- **Justificación:** En entornos de producción modernos, las aplicaciones web suelen escalar horizontalmente, es decir, se ejecutan múltiples instancias de la aplicación en diferentes servidores o contenedores de Docker simultáneamente. Al utilizar `WithMachineName()`, Serilog inyecta automáticamente el nombre del equipo que procesó la solicitud en cada log. Esto es vital para el equipo de seguridad y operaciones, ya que permite aislar rápidamente qué servidor específico está fallando o siendo atacado, sin tener que programar esta información manualmente en cada controlador.